

RWD 分析を R で

西川 寛来

2026-04-08

スライドはこちら

- https://nissinbo.github.io/RwPL_1_rwd/



自己紹介



- 西川寛来
 - 製薬業界で修行中
 - LinkedIn
 - GitHub

こんな話をします 🙌

- RWD 分析におけるデータハンドリングの tips
- データベース研究で便利なパッケージ
- OMOP CDM の紹介

RWD 分析の大変なところ 🤔

製薬企業でよく利用される RWD

- 医療情報データベース
 - レセプトデータ
 - 電子カルテデータ

解析上の難しい点・勘所

- 大規模データのハンドリング
- 治療の連続性 (処方継続、中止、切り替え) の追跡
- ICD-10 等のコード体系

結論: これで学ぼう



- 疫学のための R ハンドブック
 - 疫学データ解析の実践ガイド
 - 可視化・統計・レポーティングなど幅広くカバー

これが最速だと思います。が、本発表ではもっとマニアックな tips をお伝えします

基本的な操作

非等価結合 (Non-equi joins)

- RWD 研究で頻出のシチュエーションで便利
 - 処方日より前の直近の検査値を取得したい
 - 診断日から n 日以内の入院を探したい

```
1 # 処方テーブル
2 rx <- tibble(patient_id = c(1, 1, 2), rx_date = as.Date(c("2023-01-15", "2023-06-01", "2023-03-10")))
3
4 # 検査値テーブル
5 lab <- tibble(
6   patient_id = c(1, 1, 1, 2),
7   lab_date   = as.Date(c("2022-12-01", "2023-01-10", "2023-05-20", "2023-02-15")),
8   hba1c      = c(7.2, 7.5, 6.8, 8.1)
9 )
10
11 # 処方日より前の直近の検査値を非等価結合で取得
12 result <- rx >
13   left_join(lab, join_by(patient_id, closest(rx_date >= lab_date)))
```

- `left_join()` → `filter()` → `arrange()` → `slice_head()` と同様の処理が、`join_by()` 1 行で完結！

治療継続性 (Persistence) の定義

e.g., 「治療が 15 日以上途切れたら中断とみなす」 みたいなルールを定義したい

```
1 rx <- tribble(  
2   ~patient_id, ~start_date, ~end_date,  
3   1,           "2024-01-01", "2024-04-01",  
4   1,           "2024-01-15", "2024-02-01",  
5   1,           "2024-02-15", "2024-03-01"  
6 )  
7 mutate(start_date = as.Date(start_date), end_date = as.Date(end_date))  
8  
9 # 各処方方を lubridate の interval 形式に変換  
10 rx_intervals <- rx  
11   arrange(patient_id, start_date)  
12   group_by(patient_id)  
13   mutate(  
14     rx_interval      = interval(start_date, end_date),  
15     running_max_end  = accumulate(end_date, max),  
16     prev_coverage    = lag(interval(first(start_date),  
17                               running_max_end))  
18   )
```

①

②

大規模データへの対応

あなたのデータ環境は？

- DB が用意されている → SQL で抽出すれば OK
 - SQL または `dbplyr`
- DB がない / R 上で大きなデータを扱う → DuckDB
 - `duckplyr`

DB があるなら: SQL, dbplyr どちらもOK

```
1 library(DBI)
2 library(dplyr)
3 library(dbplyr)
4
5 con ← dbConnect(RPostgres::Postgres(), ... ) # DB接続
```

SQL

```
1 sql ← "
2 SELECT patient_id, drug, COUNT(*) AS n_rx
3 FROM rx
4 WHERE start_date ≥ DATE '2023-01-01'
5     AND start_date < DATE '2024-01-01'
6 GROUP BY patient_id, drug
7 "
8
9 n_rx ← dbGetQuery(con, sql)
```

dbplyr

```
1 n_rx ← con ▷
2   tbl("rx") ▷
3   filter(
4     between(
5       start_date,
6       as.Date("2023-01-01"),
7       as.Date("2024-01-01")
8     )
9   ) ▷
10  count(patient_id, drug) ▷
11  collect() # ここで初めて R に持ってくる
```

それでも「R 上で大きなデータ」を扱いたい場面がある

- ウチには DB なんてないよ！ CSV ファイルでしかデータもらえないよ！
- DB はあるけど、権限の都合で DB 上に一時テーブルが作れない

そんな時には `duckplyr`

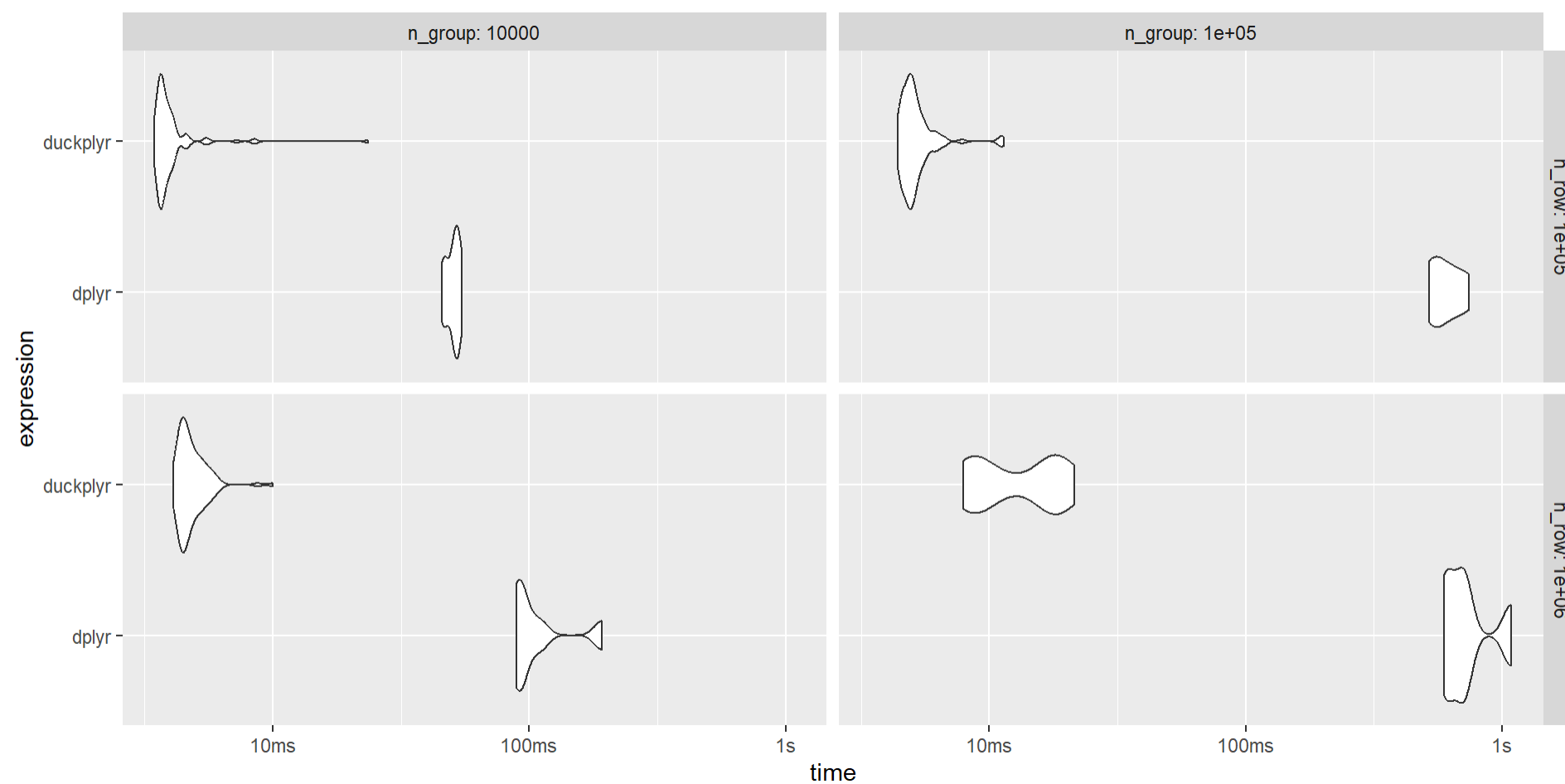
duckplyr のうれしさ

- DB を準備しなくても、R 上で大規模データを扱える
- 速い
- **duckplyr** 記法で書ける

```
1 library(duckplyr)
2
3 penguins_duck ← penguins ▷
4   as_duckdb_tibble() ▷ # duckplyr data frame に変換
5   group_by(species) ▷ # 普通に dplyr を書くだけ
6   summarise(mean_bill_length = mean(bill_len, na.rm = TRUE))
```

dplyr と duckplyr の速度比較

► Code

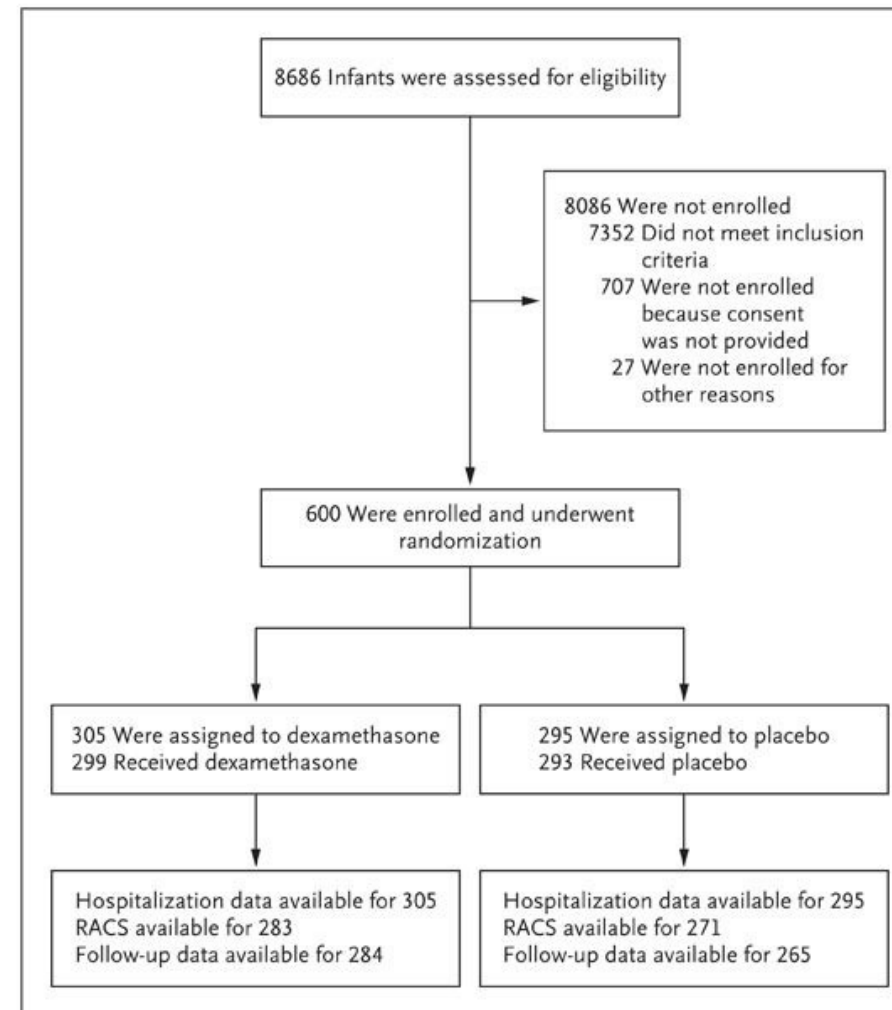


- 速いです

フローチャート作成

CONSORT diagram とは

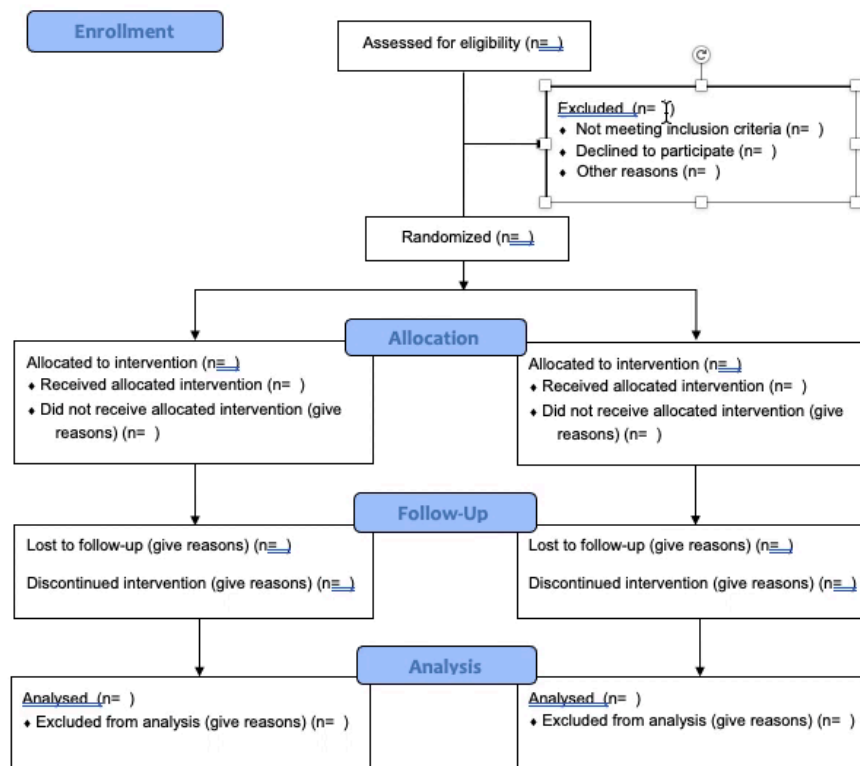
- 臨床研究において患者選択や割り当ての流れを示したフローチャート
- 集団選択が複雑な RWD 研究ではとても重要！



よくある CONSORT diagram の作り方



CONSORT 2010 Flow Diagram



- コピペ、体裁を整える作業で消耗するほど人生は長くない
- データと紐付けて、患者数を自動で入れたい。どうせ明日には除外基準が1つ追加されて、患者数も変わるんだし...
- 図はすべて **ggplot2** で書きたい

CONSORT diagram を作るための R パッケージ

- flowchart
 - “Tidy Flowchart Generator”
 - `dplyr` っぽく piping してフローチャートを作成

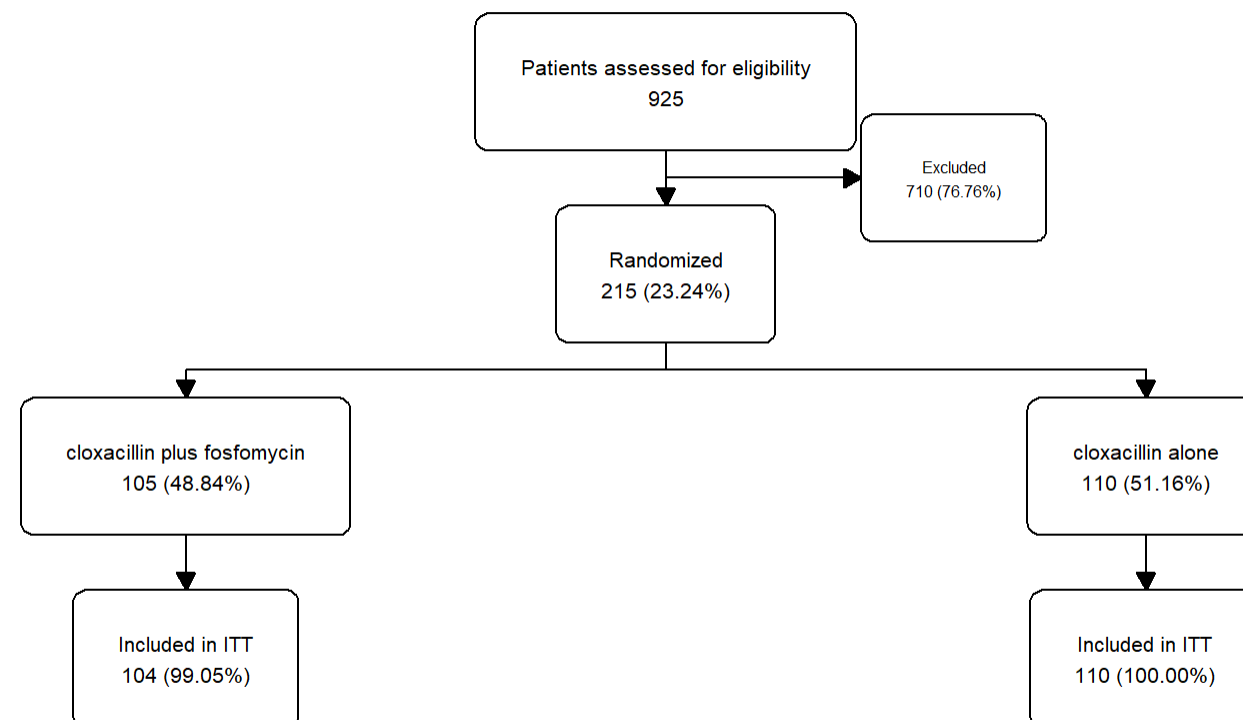


flowchart の使い方

```

1 safo ▷
2   as_fc(label = "Patients assessed for eligibility") ▷
3   fc_filter(!is.na(group), label = "Randomized", show_exc = TRUE) ▷
4   fc_split(group) ▷
5   fc_filter(itt = "Yes", label = "Included in ITT") ▷
6   fc_draw()

```



- 患者集団やコードセットに変更があっても、患者数のコピペは不要！

まとめ

RWD 分析を R でやっていきましょう

基本的な操作

- `join_by()` を使った非等価結合は便利
- 治療継続性は `lubridate` の `interval` と `accumulate()` を組み合わせると確実

大規模データ

- DB があるなら `SQL / dbplyr`
- ローカルの R でも `duckplyr` を使うだけで高速化！

フローチャート作成

- 全部 R でやって、コピペ作業から解放

Enjoy 🙌